

Performance Measurement of a Web Application

CS 681: Assignment 1

Analysis of Open and Closed Queueing Systems

Sameer Arvind Patil(23b1035) Harsh Suthar(23b1067)

February 14, 2026

Outline

- 1 Problem Statement
- 2 Open load system
- 3 Hardware & Software
- 4 Experimental Results
- 5 BONUS
- 6 Conclusion
- 7 Closed Load System
- 8 Hardware & Software
- 9 Experimental results
- 10 Conclusion

Problem Statement

- Measure and analyze the performance of a web server using open and closed load models.
- Compare observations with queueing theory (Little's law, Utilization law) and plot relevant metrics with load on server.

Open Load System

Client, Server, and Network specifications

- Server: HP Pavilion, Ubuntu 24.04.3, single core, 8GB RAM
- Client: Macbook Air M1, MacOS Sonoma, single core, 8GB RAM
- Network: WiFi-100 Mbps
- Web server: Apache
- PHP
- Tools: curl (C++) and mpstat

System Setup Diagram

Simple network diagram: Client (curl) — Network — Web Server
(Apache+PHP)

Theoretical Predictions

- Single-server queuing system
- Predictions of metrics
 - Response Time - Constant then linear increase
 - Server Utilization - Linear increase and eventual saturation
 - Throughput - Linear increase and eventual saturation
- Utilization Law Prediction - Constant service time across the runs

Experiment Parameters

The Open-load experiment has been simulated using a C++ script.

- *Arrival rates chosen*: From 10 to 200, at an interval of 10. 94 and 102 have also been added to get some extra datapoints in the Response vs rate plot.
- *Duration of each run*: 180 seconds(3 minutes)
- *Distribution of arrival rate*: Memoryless/exponential. Arrivals have been kept Memoryless/exponential to get better plot around the saturation point.
- *Request Timeout*: 20 seconds. Incase the request is dropped anywhere in the network, and not at the server
- *VAL in test.php*: \$val is set to 6,000,000 for this experiment.

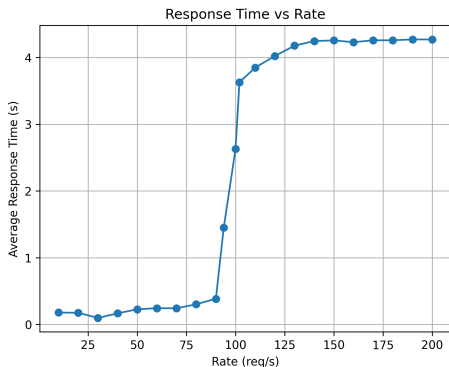
Experimental Results

Data Points

The readings obtained from the simulation:

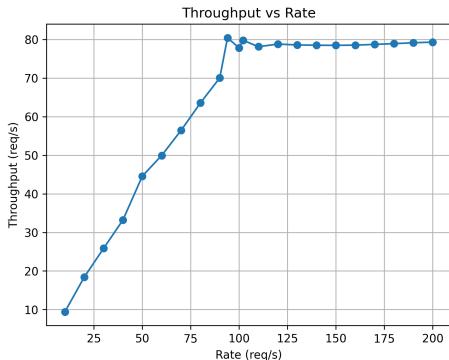
Rate	Completed	Failed	Throughput (req/s)	Avg Latency (s)	Server util. (%)
10	1694	0	9.41	0.179	25.21
20	3313	0	18.41	0.175	39.16
30	4656	0	25.87	0.098	49.28
40	5981	0	33.23	0.167	54.65
50	8025	0	44.58	0.227	63.83
60	8989	0	49.94	0.244	68.18
70	10156	0	56.42	0.243	73.03
80	11438	0	63.54	0.303	80.49
90	12608	0	70.04	0.385	87.74
94	14477	0	80.43	1.451	89.33
100	14004	193	77.80	2.629	94.40
102	14364	1228	79.80	3.630	94.64
110	14062	1045	78.12	3.849	95.03
120	14185	2152	78.81	4.022	94.88
130	14145	3981	78.58	4.177	94.74
140	14130	6531	78.50	4.247	96.16
150	14126	7135	78.48	4.256	95.69
160	14136	6955	78.53	4.229	95.60
170	14171	7981	78.73	4.258	96.58
180	14205	11742	78.92	4.259	96.21
190	14247	13404	79.15	4.271	96.37
200	14275	12036	79.31	4.270	95.82

Response Time vs Request Arrival Rate



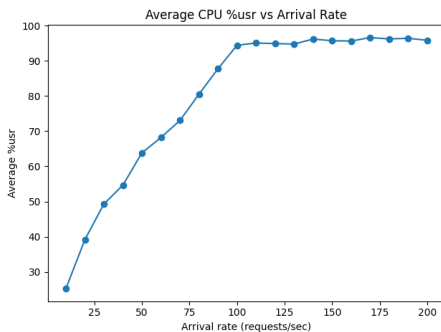
- Initially the response rate is almost constant, with a slight increase (probably due to increased chance of 2 requests coming together, leading to a bit of queueing even when service rate is faster than arrival rate). It also incorporates the network delay part
- Once the server starts saturating at around $\lambda \approx 100$, $\lambda > \mu$, the response rate sees a sudden jump, and becomes constant at around 4.27s, which is theoretically equal to $(K + 1) \cdot \tau$.

Throughput vs Request Arrival Rate



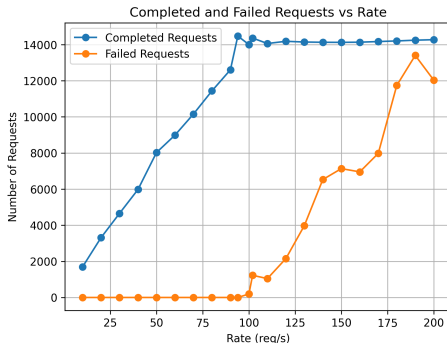
- Initially, throughput increases linearly with the arrival rate λ , as the service rate μ is greater than the arrival rate, leading to zero failures.
- Once the arrival rate exceeds the service rate at around $\lambda \approx 100$, we can visibly see the throughput flatten out to a constant value of $\Lambda = 79 \text{ req/sec} = 1/\mu$. The failure rate keeps on increasing after this point.

Server Utilization vs Request Arrival Rate



- This curve is pretty much identical to the throughput curve, due to the utilization law, validating theoretical predictions.
- This curve also flattens out (Server utilization = 100%) at an arrival rate of around 100.

Unsuccessful Requests vs Request Arrival Rate



- This curve also matches theoretical predictions. When the server was able to handle the incoming requests, we can see that the failure rate was 0.
- Once the server reaches saturation, we can see that the completed request in the 3 minute period constants out at 14000.
- After this point, all extra requests are failing due to queue length being reached at the server.

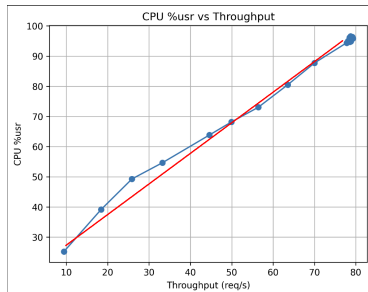
Estimating Service Time - Utilization Law

Core relation (Utilization law):

$$\rho = \Lambda \cdot \tau \quad \Rightarrow \quad \boxed{\tau = \frac{\rho}{\Lambda}}$$

When server utilization reaches saturation, we can take $\rho = 1$, giving

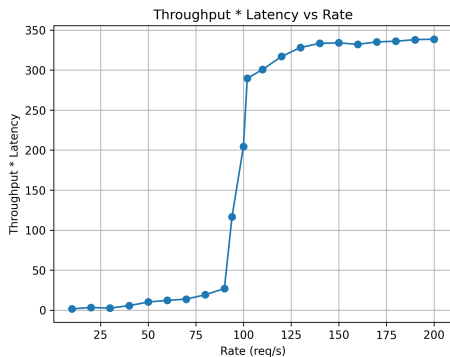
$$\tau = 1/\Lambda = 1/79.15 = 12.61ms$$



Little's Law

Little's Law:

$$N = \Lambda \cdot R$$



When the server saturates, we can see that the effect on N is visible, as it becomes almost constant at around 338, giving us a queue size of

$$K \approx 337$$

BONUS PART

BONUS PART: verifying number of people in the system

To verify the queue size obtained from little's law in saturated condition, we monitored the number of threads in the system, which corresponds to the queue length.

```
saneer@saneer-HP-Pavilion-Laptop-15-eg2xxx:~/Desktop$ while true; do busy=$(  
curl -s http://localhost/server-status?auto | awk '/BusyWorkers/ {print $2}');  
queue=$(ss -ltn sport = :80 | awk 'NR>1 {print $2}'); ts=$(date +"%H:%M:%  
S"); echo "ts Busy=$busy Waiting=$queue Total=$((busy+queue))"; slee  
p 1; done  
22:27:12 Busy=134 Waiting=209 Total=343  
22:27:15 Busy=169 Waiting=181 Total=350  
22:27:18 Busy=198 Waiting=150 Total=348  
22:27:20 Busy=212 Waiting=133 Total=345  
22:27:23 Busy=233 Waiting=120 Total=353  
22:27:25 Busy=244 Waiting=99 Total=343  
22:27:28 Busy=256 Waiting=96 Total=352  
22:27:30 Busy=256 Waiting=96 Total=352  
22:27:32 Busy=256 Waiting=95 Total=351  
22:27:34 Busy=256 Waiting=95 Total=351  
22:27:36 Busy=256 Waiting=89 Total=345  
22:27:38 Busy=256 Waiting=94 Total=350  
22:27:40 Busy=256 Waiting=84 Total=340
```

- The 'Busy workers' corresponds to the workers currently in the Apache server, being either processed, or currently in the inner queue.
- As the Apache server has a limit of 256 concurrent threads, some of the remaining requests move to the system queue, which has another ≈ 100 requests.

This gives a total of ≈ 345 requests in the system at any point. This number is close to the obtained queue length of $K = 337$.

Conclusion

Metric	Measured	Using Derived results
Throughput Λ_{exp}	79.15 req/s	—
Mean service time τ_{exp}	12.61 ms/req (0.0126 s)	—
Queue Length (K)	344 requests	337 requests
Max response time(R)	4.27sec	4.24sec ($(K_{\text{theoretical}} + 1) \cdot \tau$)

Closed Load System

Client, Server, and Network specifications

- Server: HP Pavilion, Ubuntu 24.04.3, single core, 8GB RAM
- Client: Macbook Air M1, MacOS Sonoma, single core, 8GB RAM
- Network: WiFi-100 Mbps
- Web server: Apache
- PHP
- Tools: C++ curl library, the client shall be sending requests via multithreading and a c++ script

Theoretical Predictions

Metrics:

- Response Time: Follows low load asymptote initially, then increases linearly
- Server Utilization: Increases linearly until near saturation
- Throughput: Increases linearly until saturation

Utilization Law Prediction:

Constant service time across the runs

Little's law:

Think Time = $M/\Lambda - R_{sys}$

Experiment Parameters

The Closed-load experiment has been simulated using a C++ script at client side.

- *Arrival rates chosen*: From 10 to 300, at an interval of 20.
- *Think Time(γ)*: 4 seconds.
- *Duration of each run*: 180 seconds(3 minutes)
- *Request Timeout*: 30 seconds. In case the request is dropped anywhere in the network, and not at the server
- *VAL in test.php*: \$val is set to 12,000,000 for this experiment.

The clients have been staggered at the beginning of the experiment to prevent bursts of requests at the start.

Experimental Results

Data Points

The readings obtained from the simulation:

N	Throughput	Resp. Time	Failures	Successes	Util (%)
10	2.40	0.034	0	432	8.68
30	7.50	0.041	0	1350	20.99
50	12.24	0.051	0	2204	35.18
70	16.89	0.071	0	3040	60.04
90	22.12	0.103	0	3982	67.79
110	24.40	0.185	0	4391	75.07
130	28.88	0.407	0	5198	89.41
150	27.93	0.853	0	5027	97.36
170	31.25	1.427	0	5625	98.87
190	31.15	2.238	8	5607	99.68
210	31.25	2.292	34	5625	97.90
230	30.01	3.026	67	5402	95.03
250	31.25	3.482	79	5625	96.87
270	29.48	4.561	124	5306	96.33
290	31.24	5.666	195	5623	93.96

Response Time vs Number of Users

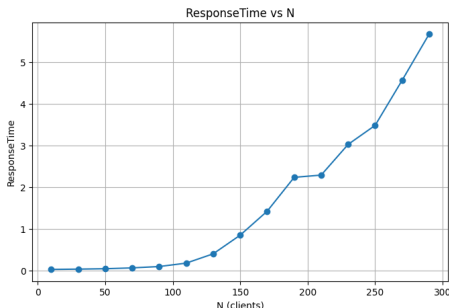


Figure: Response Time vs M

- For small M the response time stays near the mean service time S , pretty flat. It is a bit more than the service time exactly, probably due to network latency in the system also being counted in the response time.
- When M approaches the experimental saturation $M^* \approx 135$ (knee), queueing begins to appear.
- Beyond M^* response time grows linearly as the queue builds.

Experimental vs Theoretical Kleinrock saturation number

Theoretical:

$$\hat{M}^* = 1 + \frac{\gamma}{\tau} = 1 + \frac{4000}{33} = 126$$

Experimental(from the plot):

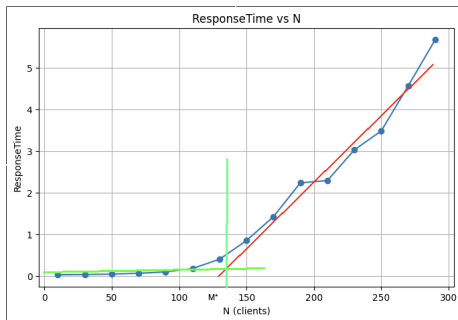
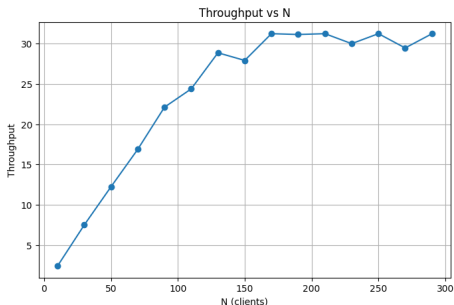


Figure: M^* from the plot ≈ 135

Throughput vs Number of Users



The throughput increases linearly up till around 150 users, after which it becomes almost constant at around 32 *req/sec*, giving an experimental service time of

$$\tau \approx 33 \text{ ms/req}$$

Some the non-smooth nature of this curve is probably due to various factors like other processes on the server machine, other IO processes etc.

Server Utilization vs Number of Users

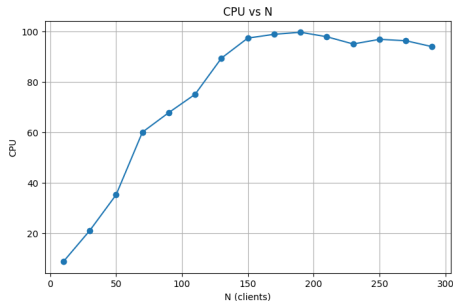


Figure: Server utilization vs number of users

- The Server utilization follows an identical trend as the throughput plot (due to the utilization law)
- There is some disparity between Server utilization and throughput plot, we believe its mostly due to other processes running on the server laptop, and some IO factor in the OS side of things.

Failures vs Number of Users

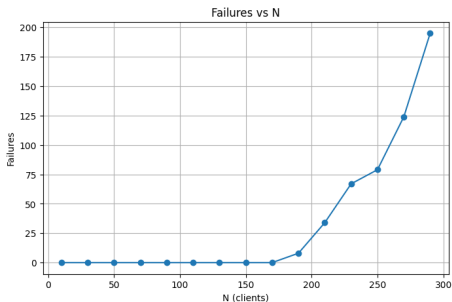


Figure: Failures vs Number of users

Failure count starts increasing once the server saturates out, and increases linearly as N increases.

Response Time vs Throughput

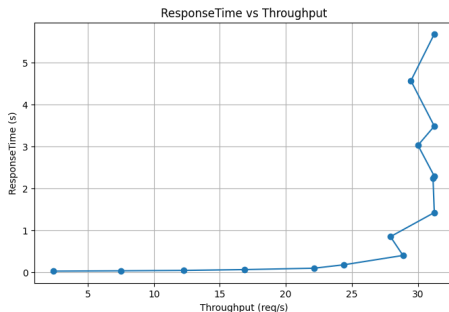


Figure: Response Time vs Throughput

- At low offered load throughput rises while response time stays near the mean service time S .
- As throughput approaches the server capacity $X_{\max} \approx 1/S$ a “knee” appears: throughput growth slows and response time begins to increase.
- In the saturated region throughput plateaus (service-limited) and further load mainly increases response time due to queueing.

Server Utilization vs Throughput

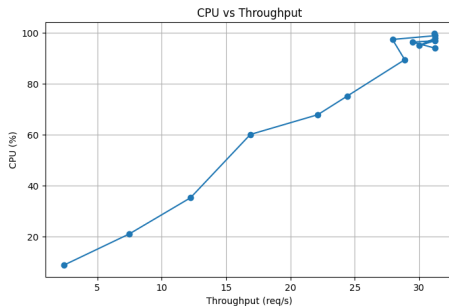


Figure: Server Util. vs Throughput

This forms a straight line, with slope = $\tau = 30ms/req$ till server doesn't saturate out, after which both throughput and server util. is constantly equal to their maximum.

Estimating Service Time - Utilization Law

Core relation (Utilization law):

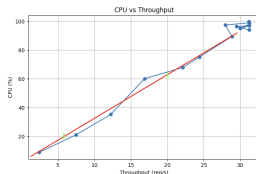
$$\rho = \Lambda \cdot \tau \quad \Rightarrow \quad \boxed{\tau = \frac{\rho}{\Lambda}}$$

We can find the service time by 2 approaches, validating the experimental setup,

- slope of the Server utilization vs Throughput plot
- $1/\Lambda$, when the throughput has flattened out in the graph

Compute (brief):

① Result from method 1: $\tau = 30ms/req$



② Result from method 2: $\tau = 33ms/req$

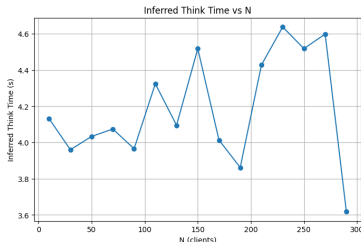
Little's Law — Think Time

$$N = \Lambda (R + \gamma)$$

where N is number of users, Λ throughput (req/s), R response time (s), and γ think time (s). **Configured value of think time: 4 seconds**

Inferred think time:

$$\hat{\gamma} = \frac{N}{\Lambda} - R = 4.2sec$$



Conclusion

Metric	Measured	Using derived results
Throughput Λ_{exp}	32 req/s	—
Kleinrock saturation M^*	135	$1 + \frac{\gamma}{\tau_{\text{exp}}} = 1 + \frac{4}{0.033} \approx 126$
Inferred think time $\hat{\gamma}$	4.2 s	4 s (Configured)
Service time	30 ms (slope)	$\approx 33 \text{ ms}$ (from $1/\Lambda_{\text{max}}$)

Experiments scripts

The scripts for this experiment can be found in the following github repo:

https://github.com/SameerPatil20/CS681-just_better